How Incremental Update Works

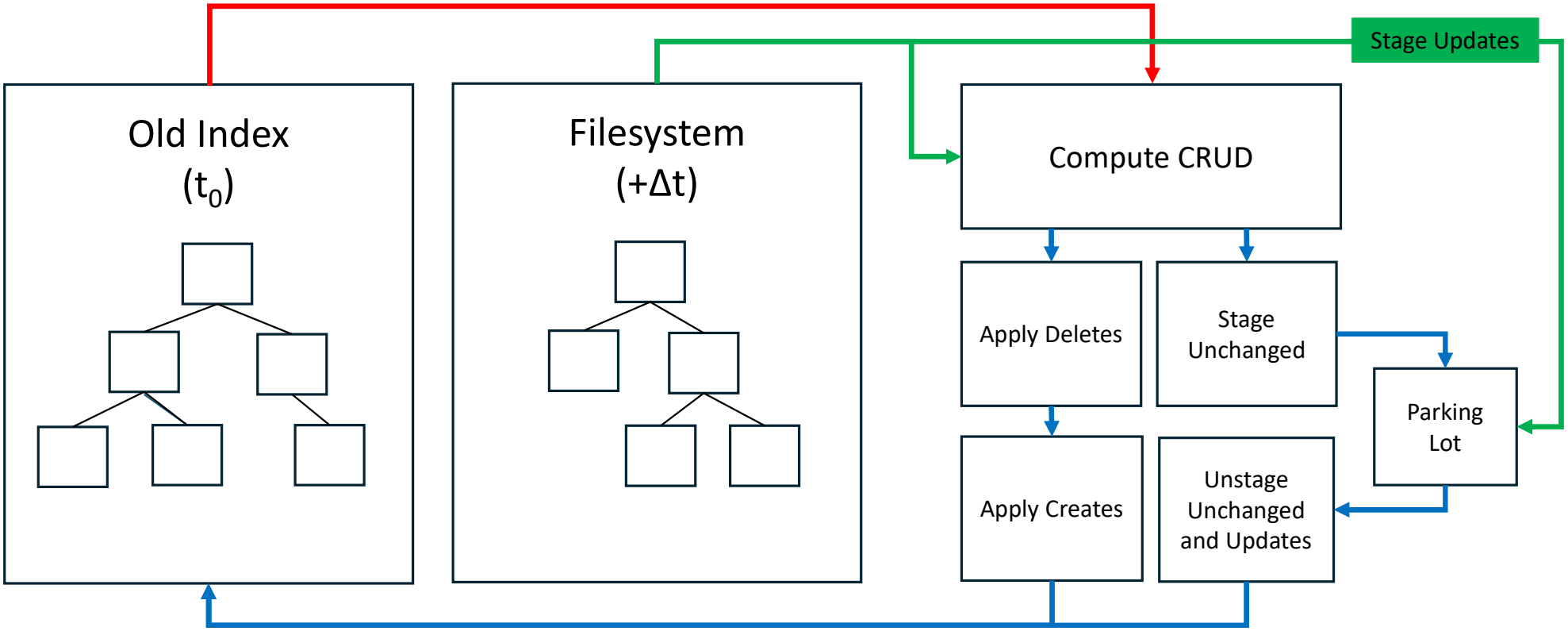
Parking Lot Method and Grand Vision

Gary Grider
Jason Lee

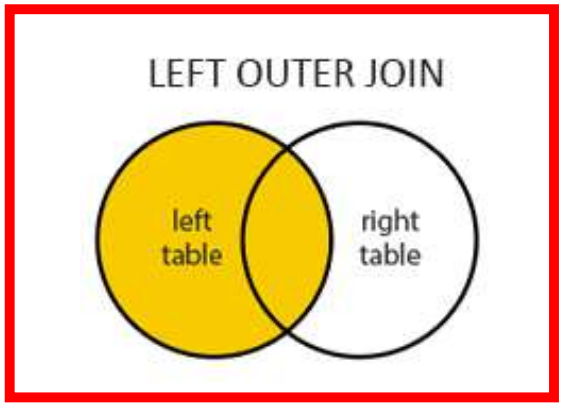
LA-UR-26-25065

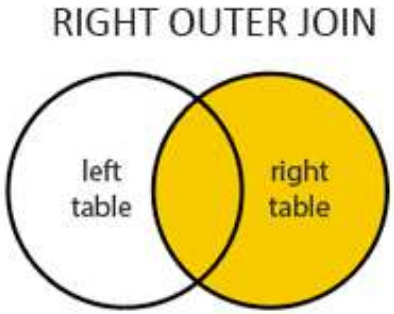
Parking Lot Method

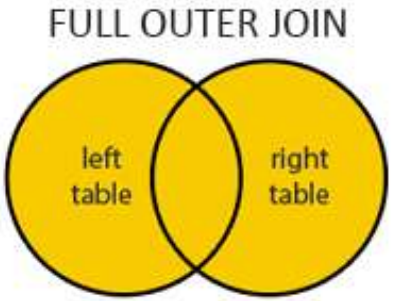
Incremental Updates



SQLite join background







SQLite does not have RIGHT [OUTER] JOIN

https://www.dofactory.com/sql/outer-join

bfwreaddirplus2db

- bfw → breadth first walk
- readdirplus → using `readdir(3)` to get `d_type`/`st_mode` (without `stat(2)`)
- 2db → write changes to dbs

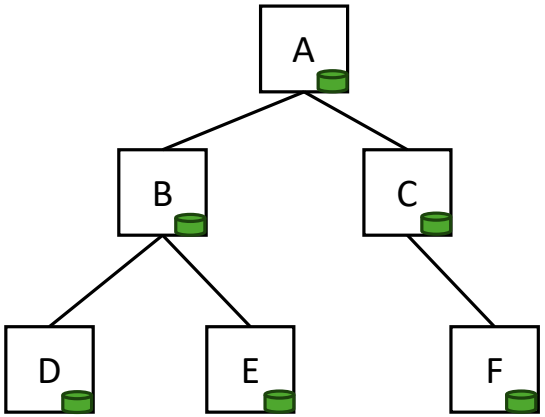
Incremental Walk

- bfwreaddirplus2db
  - Scan source tree
    - Generate db containing ALL directories and if they are suspect
    - Create update dbs for suspected changes and place them into the parking lot

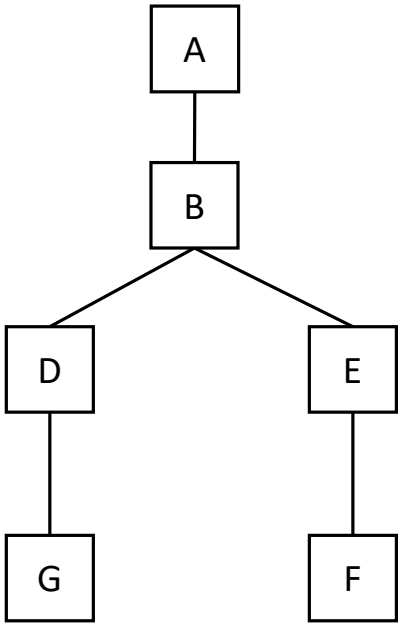
- gitest.py
  - Scan index
    - Generate db containing ALL directories
  - Reshape index
  - Move update dbs into index

Filesystem States

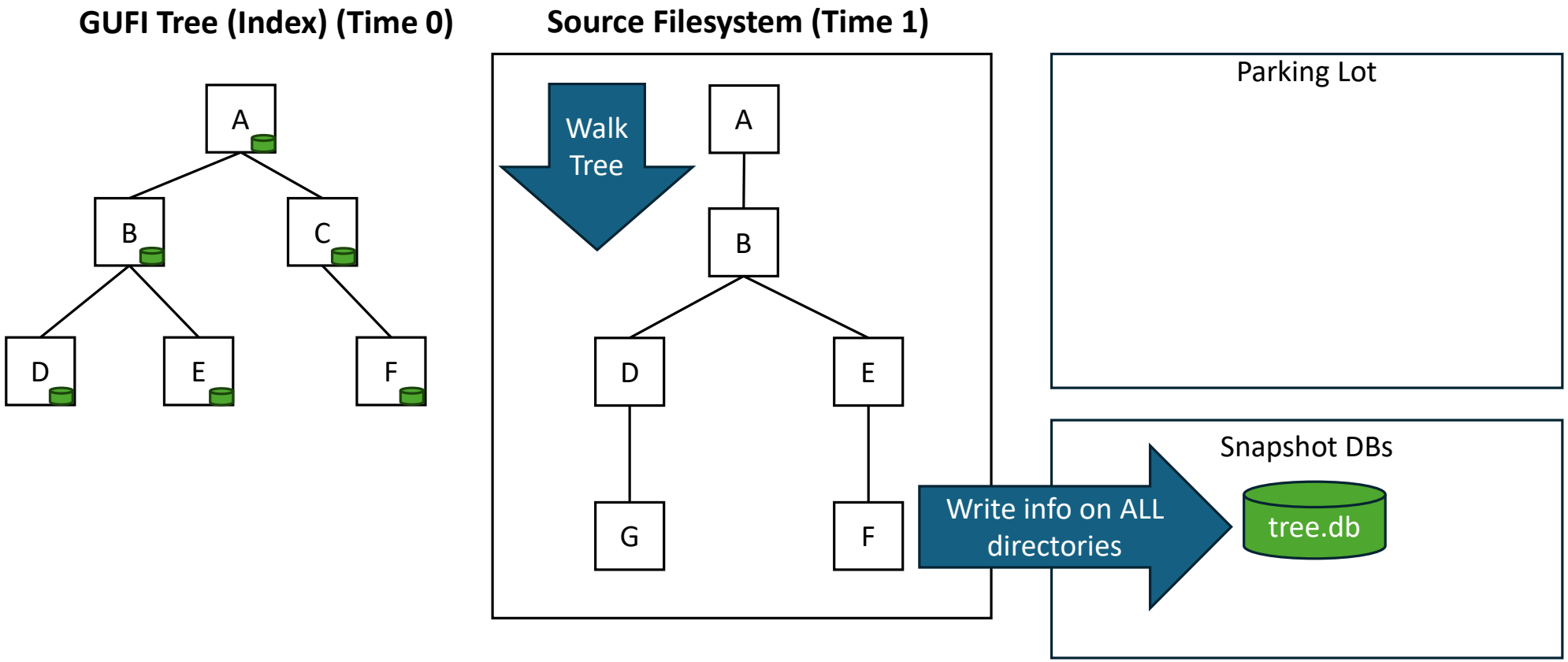
GUFI Tree (Index) (Time 0)



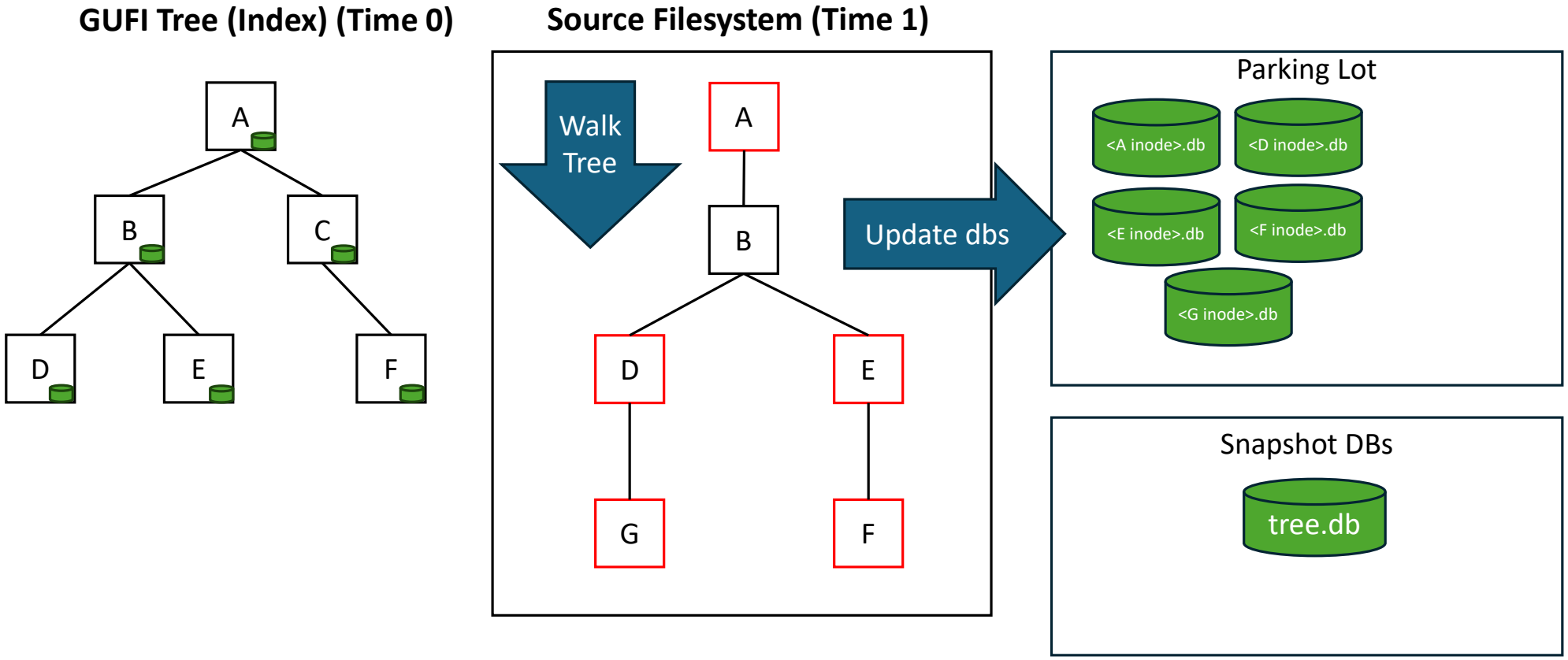
Source Filesystem (Time 1)



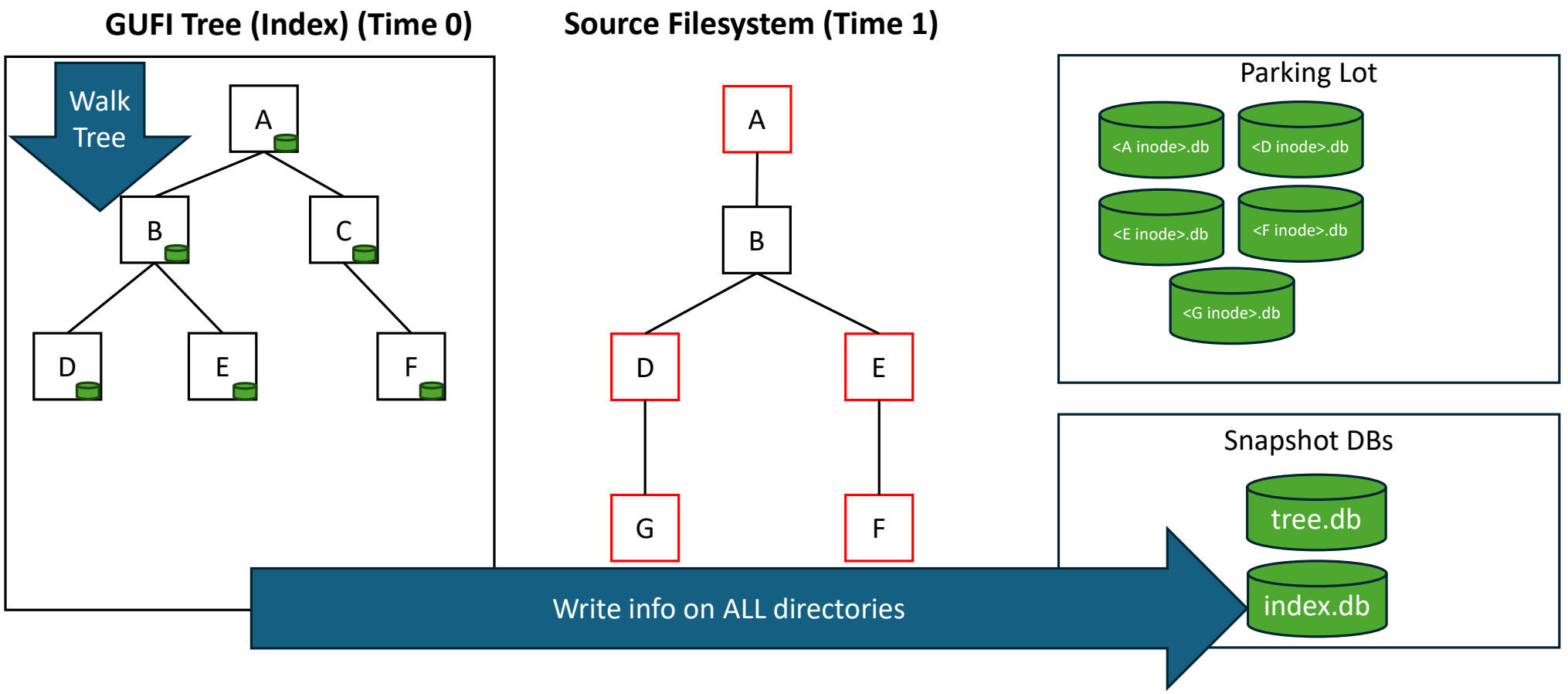
bfwreaddirplus2db: Tree Walk



bfwreaddirplus2db: Detect Suspects and Create Update DBs (same time as Tree Walk)



gitest.py: Scan Existing Index



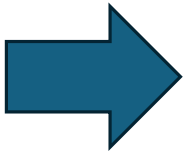
Snapshot DBs

path	inode	pinode	depth	suspect
A	A	..	0	0
B	B	A	1	0
C	C	A	1	0
D	D	B	2	0
E	E	B	2	0
F	F	C	2	0

path	inode	pinode	depth	suspect
A	A	..	0	1
B	B	A	1	0
D	D	B	2	1
E	E	B	2	1
F	F	E	3	1
G	G	E	3	1

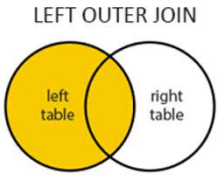
Find Changes

```
CREATE TEMP VIEW jt AS
SELECT a0.path a0path,
       a0.type a0type,
       a0.inode a0inode,
       a0.pinode a0pinode,
       a0.suspect a0suspect,
       a1.path a1path,
       a1.type a1type,
       a1.inode a1inode,
       a1.pinode a1pinode,
       a1.suspect a1suspect
FROM b0 AS a0
LEFT OUTER JOIN b1 AS a1 ON
a0.inode=a1.inode
UNION ALL
SELECT a0.path a0path,
       a0.type a0type,
       a0.inode a0inode,
       a0.pinode a0pinode,
       a0.suspect a0suspect,
       a1.path a1path,
       a1.type a1type,
       a1.inode a1inode,
       a1.pinode a1pinode,
       a1.suspect a1suspect
FROM b1 AS a1
LEFT OUTER JOIN b0 AS a0 ON
a1.inode=a0.inode
WHERE a0.inode IS NULL;
```



```
CREATE TEMPORARY VIEW jt AS
SELECT
        index.path    AS  indexpath,
        index.type    AS  indextype,
        index.inode   AS  indexinode,
        index.pinode  AS  indexpinode,
        index.depth   AS  indexdepth,
        index.suspect AS  indexsuspect,
        tree.path     AS  treepath,
        tree.type     AS  treetype,
        tree.inode    AS  treeinode,
        tree.pinode   AS  treepinode,
        tree.depth    AS  treedepth,
        tree.suspect  AS  treesuspect
FROM index.readdirplus  AS  index
LEFT OUTER JOIN tree.readdirplus AS tree ON index.inode ==
tree.inode
UNION ALL
SELECT
        index.path    AS  indexpath,
        index.type    AS  indextype,
        index.inode   AS  indexinode,
        index.pinode  AS  indexpinode,
        index.depth   AS  indexdepth,
        index.suspect AS  indexsuspect,
        tree.path     AS  treepath,
        tree.type     AS  treetype,
        tree.inode    AS  treeinode,
        tree.pinode   AS  treepinode,
        tree.depth    AS  treedepth,
        tree.suspect  AS  treesuspect
FROM tree.readdirplus AS tree
LEFT OUTER JOIN index.readdirplus AS index ON tree.inode ==
index.inode
WHERE index.inode IS NULL
```

Find Unchanged, Renames, Deletes




```
SELECT
        index.path     AS  indexpath,
        index.type     AS  indextype,
        index.inode    AS  indexinode,
        index.pinode   AS  indexpinode,
        index.depth    AS  indexdepth,
        index.suspect  AS  indexsuspect,
        tree.path      AS  treepath,
        tree.type      AS  treetype,
        tree.inode     AS  treeinode,
        tree.pinode    AS  treepinode,
        tree.depth     AS  treedepth,
        tree.suspect   AS  treesuspect
FROM index.readdirplus AS index
LEFT OUTER JOIN tree.readdirplus AS tree ON
index.inode == tree.inode
```

This ON makes sure A_{index} only matches with A_{tree} and not B_{tree}

Left index.db

path	inode	pinode	depth	suspect
A	A	..	0	0
B	B	A	1	0
C	C	A	1	0
D	D	B	2	0
E	E	B	2	0
F	F	C	2	0

tree.db Right

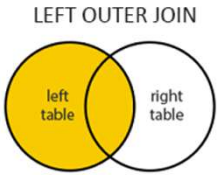
path	inode	pinode	depth	suspect
A	A	..	0	1
B	B	A	1	0
D	D	B	2	1
E	E	B	2	0
F	F	E	3	1
G	G	E	3	1

index path	index inode	index pinode	index depth	index suspect	tree path	tree inode	tree pinode	tree depth	tree suspect
A	A	..	0	0	A	A	..	0	1
B	B	A	1	0	B	B	A	1	0
C	C	A	1	0					
D	D	B	2	0	D	D	B	2	1
E	E	B	2	0	E	E	B	2	1
F	F	C	2	0	F	F	E	3	1

Left Right

Find Creates

The tables are flipped because SQLite does not have RIGHT JOIN




```
SELECT
        index.path     AS  indexpath,
        index.type     AS  indextype,
        index.inode    AS  indexinode,
        index.pinode   AS  indexpinode,
        index.depth    AS  indexdepth,
        index.suspect  AS  indexsuspect,
        tree.path      AS  treepath,
        tree.type      AS  treetype,
        tree.inode     AS  treeinode,
        tree.pinode    AS  treepinode,
        tree.depth     AS  treedepth,
        tree.suspect   AS  treesuspect
FROM tree.readdirplus AS tree
LEFT OUTER JOIN index.readdirplus AS index ON
tree.inode == index.inode
WHERE index.inode IS NULL
```

This WHERE keeps new directories

This ON makes sure A_{index} only matches with A_{tree} and not B_{tree}

Right index.db

path	inode	pinode	depth	suspect
A	A	..	0	0
B	B	A	1	0
C	C	A	1	0
D	D	B	2	0
E	E	B	2	0
F	F	C	2	0

tree.db Left

path	inode	pinode	depth	suspect
A	A	..	0	1
B	B	A	1	0
D	D	B	2	1
E	E	B	2	1
F	F	E	3	1
G	G	E	3	1

index path	index inode	index pinode	index depth	index suspect	tree path	tree inode	tree pinode	tree depth	tree suspect
A	A	..	0	0	A	A	..	0	1
B	B	A	1	0	B	B	A	1	0
D	D	B	2	0	D	D	B	2	1
E	E	B	2	0	E	E	B	2	1
F	F	C	2	0	F	F	E	3	1
					G	G	E	3	1

Right Left

Combine Changes (jt)

index path	index inode	index pinode	index depth	index suspect	tree path	tree inode	tree pinode	tree depth	tree suspect
A	A	..	0	0	A	A	..	0	1
B	B	A	1	0	B	B	A	1	0
C	C	A	1	0					
D	D	B	2	0	D	D	B	2	1
E	E	B	2	0	E	E	B	2	1
F	F	C	2	0	F	F	E	3	1

Deleted

Moved

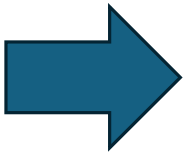
index path	index inode	index pinode	index depth	index suspect	tree path	tree inode	tree pinode	tree depth	tree suspect
					G	G	E	3	1

Created

Get Diff

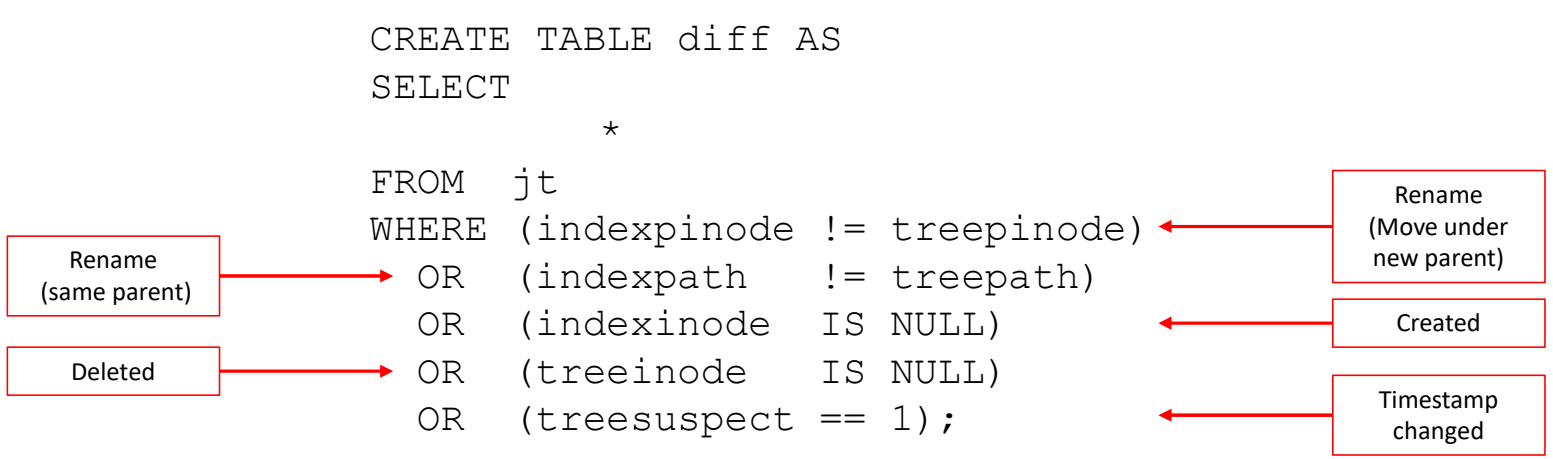
```
CREATE TABLE diff
  (
     a0path TEXT,a0type TEXT,a0inode
INT(64),a0pinode INT(64),a0suspect INT(64),
     a1path TEXT,a1type TEXT,a1inode
INT64,a1pinode INT(64),a1suspect INT(64),
     a0depth INT(64),a1depth INT(64)
  );

INSERT INTO diff
SELECT *,
       (length(a0path)-length(replace(a0path,
'/', '')))/1 AS a0depth,
       (length(a1path)-length(replace(a1path,
'/', '')))/1 AS a1depth
FROM jt
WHERE a0pinode!=a1pinode
  OR a0path!=a1path
  OR a0inode IS NULL
  OR a1inode IS NULL
  OR a1suspect=1;
```



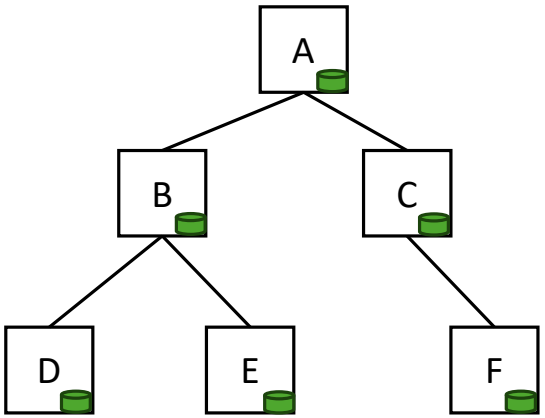
```
CREATE TABLE diff AS
SELECT
          *
FROM  jt
WHERE (indexpinode != treepinode)
  OR  (indexpath   != treepath)
  OR  (indexinode  IS NULL)
  OR  (treeinode   IS NULL)
  OR  (treesuspect == 1);
```

Get Diff

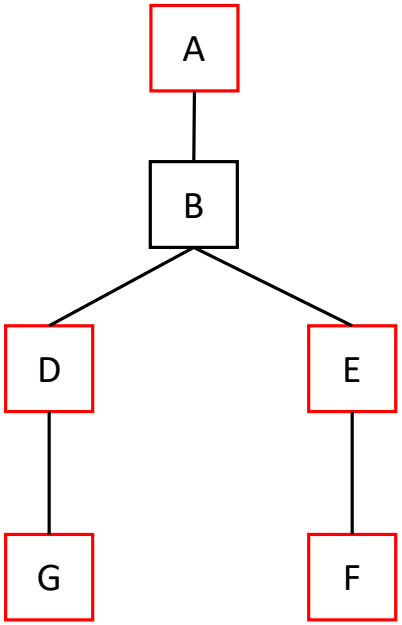

CREATE TABLE diff AS
SELECT
*
FROM jt
WHERE (indexpinode != treepinode)
OR (indexpath != treepath)
OR (indexinode IS NULL)
OR (treeinode IS NULL)
OR (treesuspect == 1);
Rename
(same parent)
Deleted
Rename
(Move under
new parent)
Created
Timestamp
changed

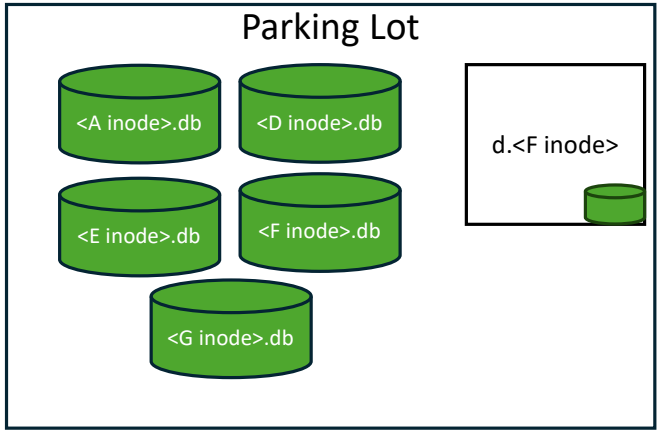
Create diff db

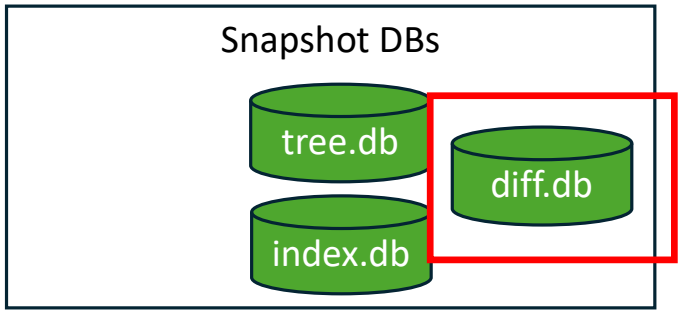
GUFI Tree (Index) (Time 0)



Source Filesystem (Time 1)







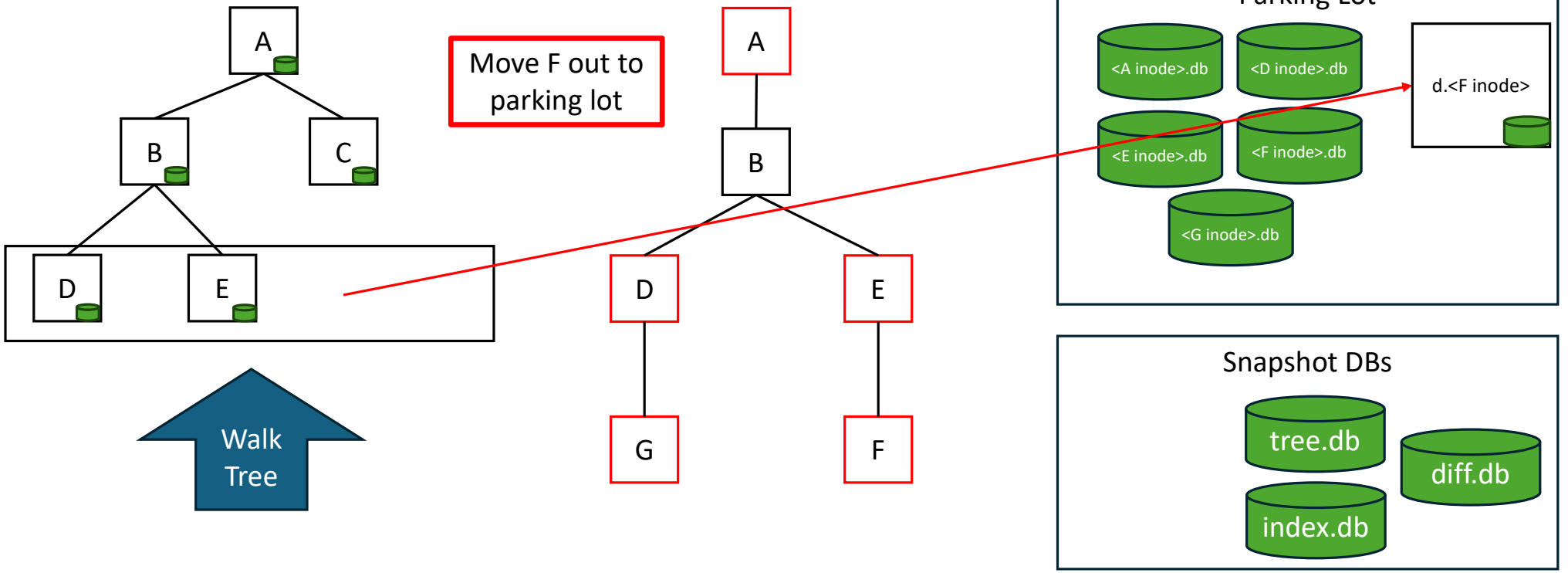
Apply Removes and Move Out

Apply Removes and Move Out

```
SELECT
    indexpath,
    treepath,
    treeinode,
    CASE WHEN  treepath IS NULL THEN 1 ELSE 0 END
FROM diff
WHERE (treepath IS NULL)
  OR  (indexpinode !=  treepinode)
  OR  (indexpath   !=  treepath)
ORDER BY indexdepth DESC;
```

GUFI Tree (Index) (Time 0)

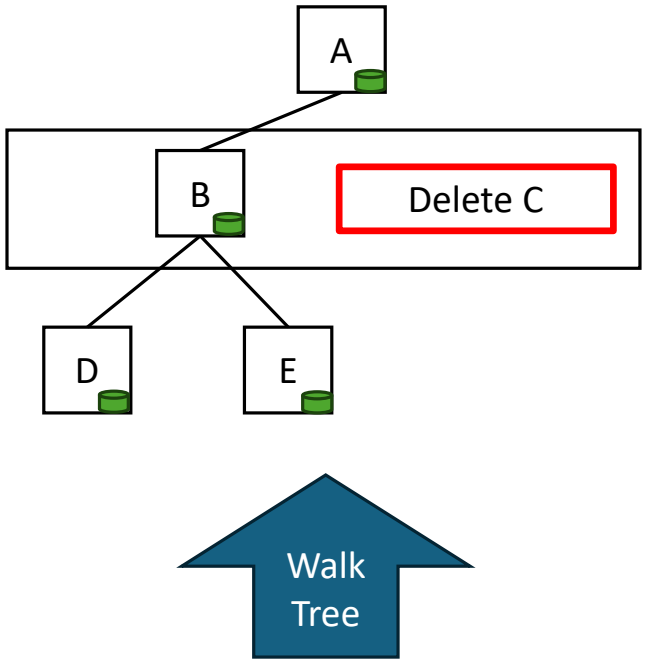
Source Filesystem (Time 1)



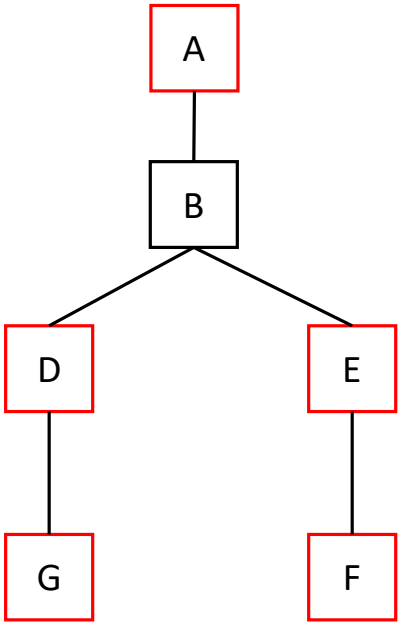
Apply Removes and Move Out

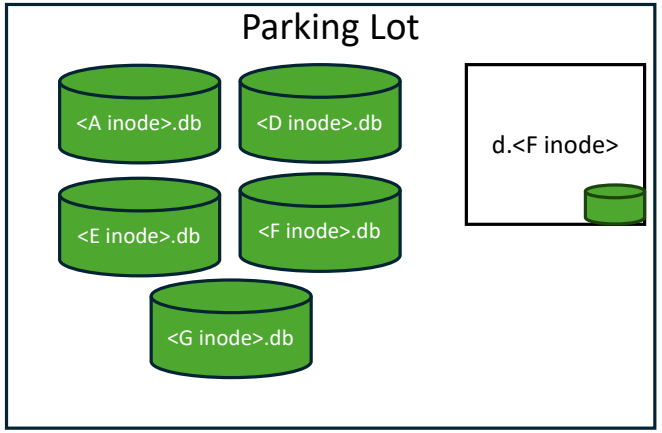
```
SELECT
    indexpath,
    treepath,
    treeinode,
    CASE WHEN  treepath IS NULL THEN 1 ELSE 0 END
FROM diff
WHERE (treepath IS NULL)
  OR  (indexpinode !=  treepinode)
  OR  (indexpath   !=  treepath)
ORDER BY indexdepth DESC;
```

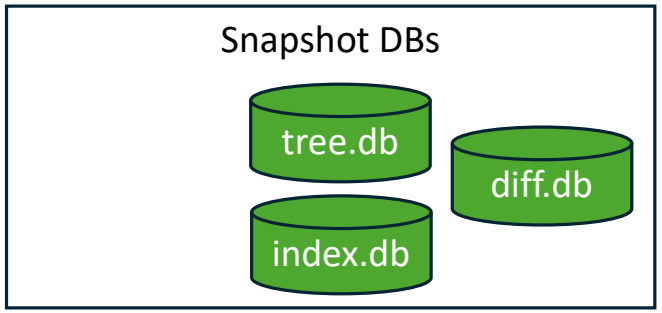
GUFI Tree (Index) (Time 0)



Source Filesystem (Time 1)





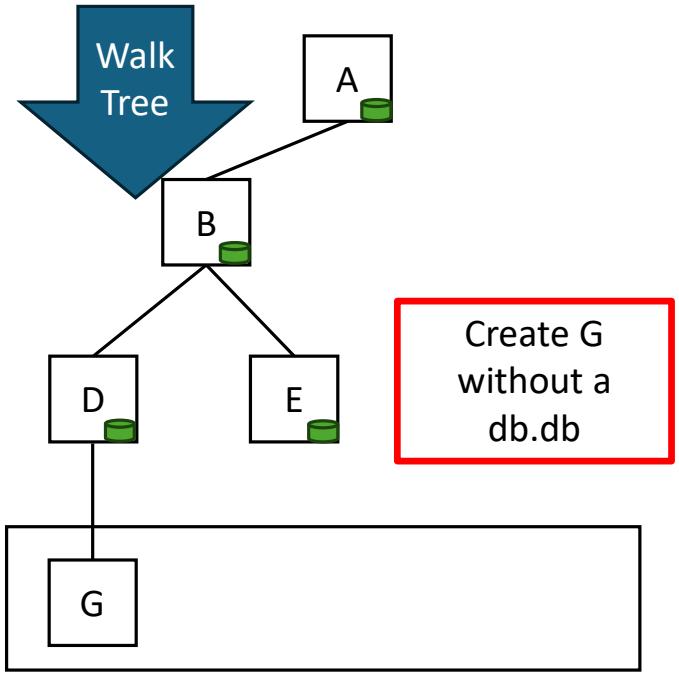


Apply Creates and Move In

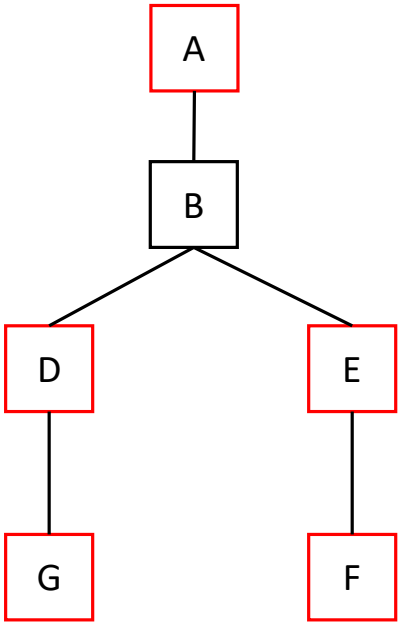
Apply Creates and Move In

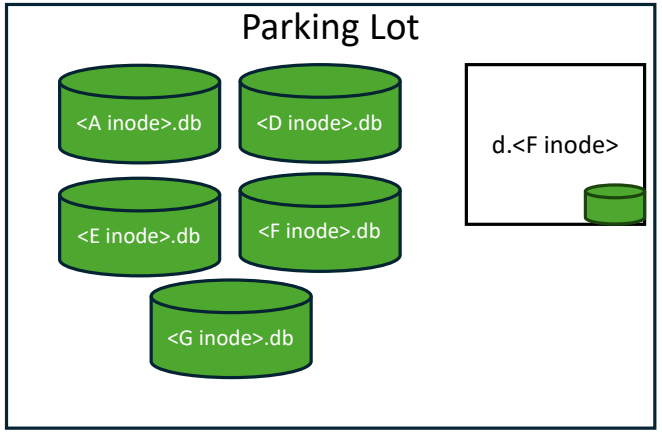
```
SELECT
    indexpath,
    treepath,
    treeinode,
    CASE WHEN  index path IS NULL THEN 1 ELSE 0 END
FROM diff
WHERE (indexpath IS NULL)
  OR  (indexpinode !=  treepinode)
  OR  (indexpath   !=  treepath)
ORDER BY treedepth ASC;
```

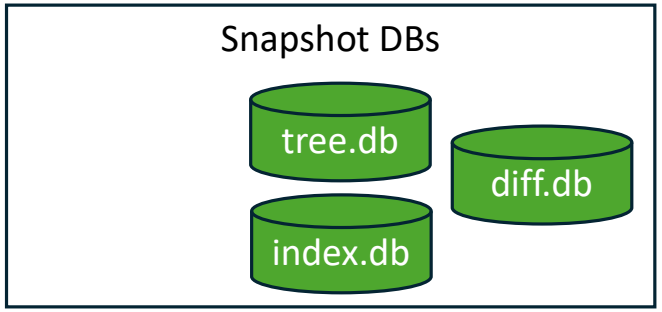
GUFI Tree (Index) (Time 0)



Source Filesystem (Time 1)

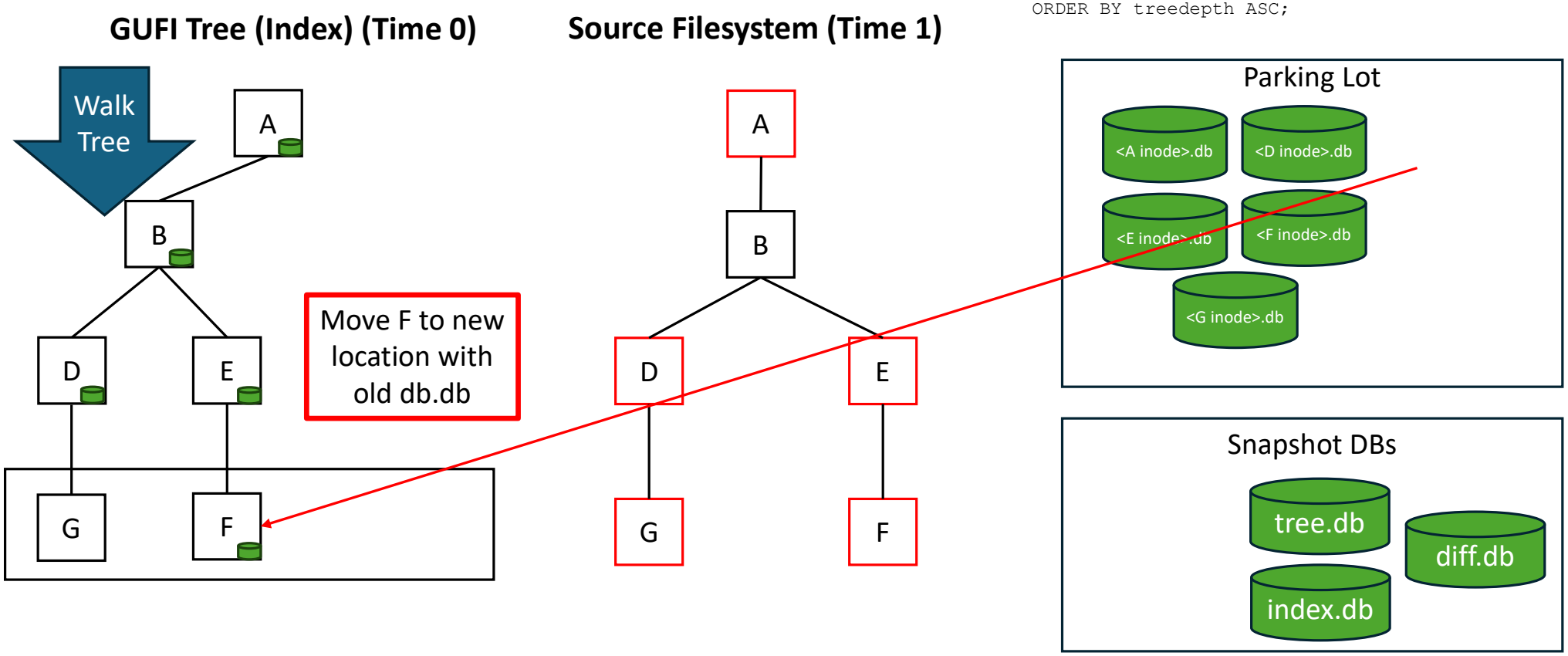






Apply Creates and Move In

```
SELECT
    indexpath,
    treepath,
    treeinode,
    CASE WHEN  index path IS NULL THEN 1 ELSE 0 END
FROM diff
WHERE (indexpath IS NULL)
  OR  (indexpinode !=  treepinode)
  OR  (indexpath   !=  treepath)
ORDER BY treedepth ASC;
```

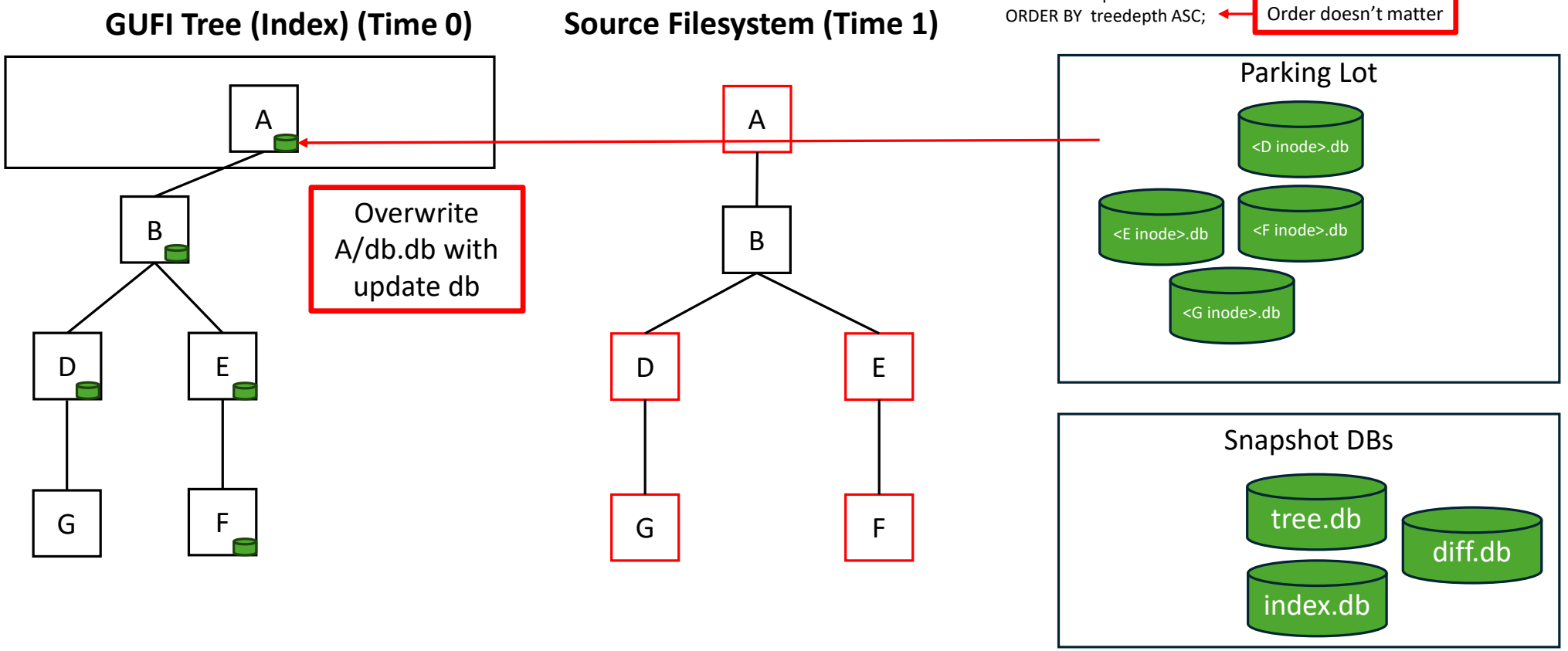


Apply db.db Updates

Apply db.db Updates

```
SELECT
        indexpath,
        tree path,
        CASE WHEN  indexpath ==  treepath THEN 0 ELSE 1 END,
        tree inode
        tree pinode,
        CASE WHEN  indexpinode ==  treepinode THEN 0 ELSE 1 END
FROM  INCR_DIFF
WHERE  treepath IS NOT NULL
ORDER BY  treedepth ASC;
```

Order doesn't matter

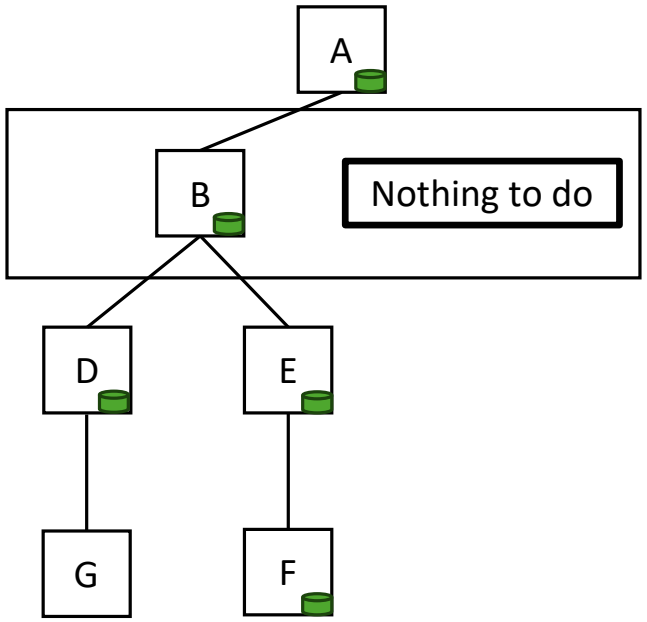


Apply db.db Updates

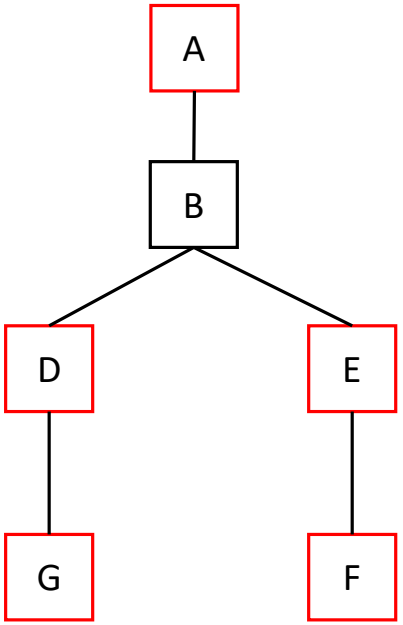
```
SELECT
        indexpath,
        tree path,
        CASE WHEN  indexpath ==  treepath THEN 0 ELSE 1 END,
        tree inode
        tree pinode,
        CASE WHEN  indexpinode ==  treepinode THEN 0 ELSE 1 END
FROM  INCR_DIFF
WHERE  treepath IS NOT NULL
ORDER BY  treedepth ASC;
```

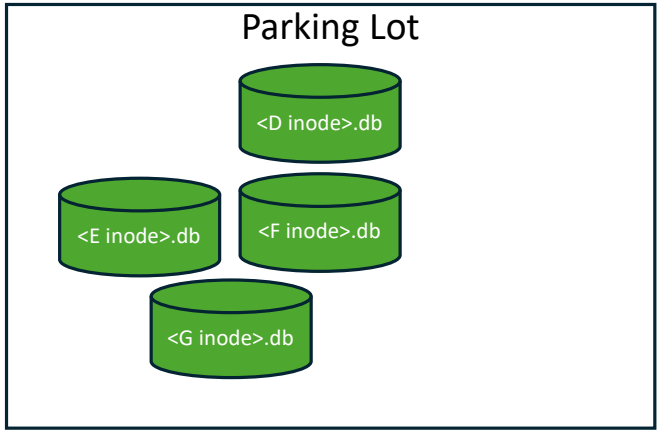
Order doesn't matter

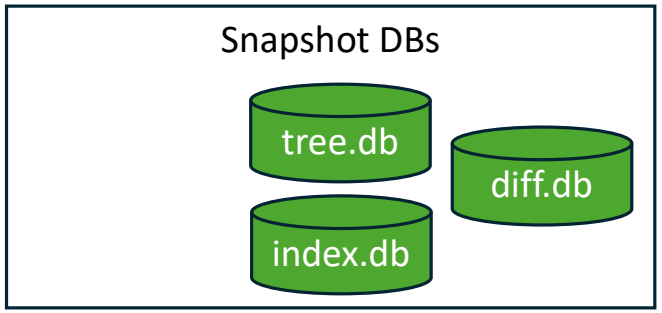
GUFI Tree (Index) (Time 0)



Source Filesystem (Time 1)







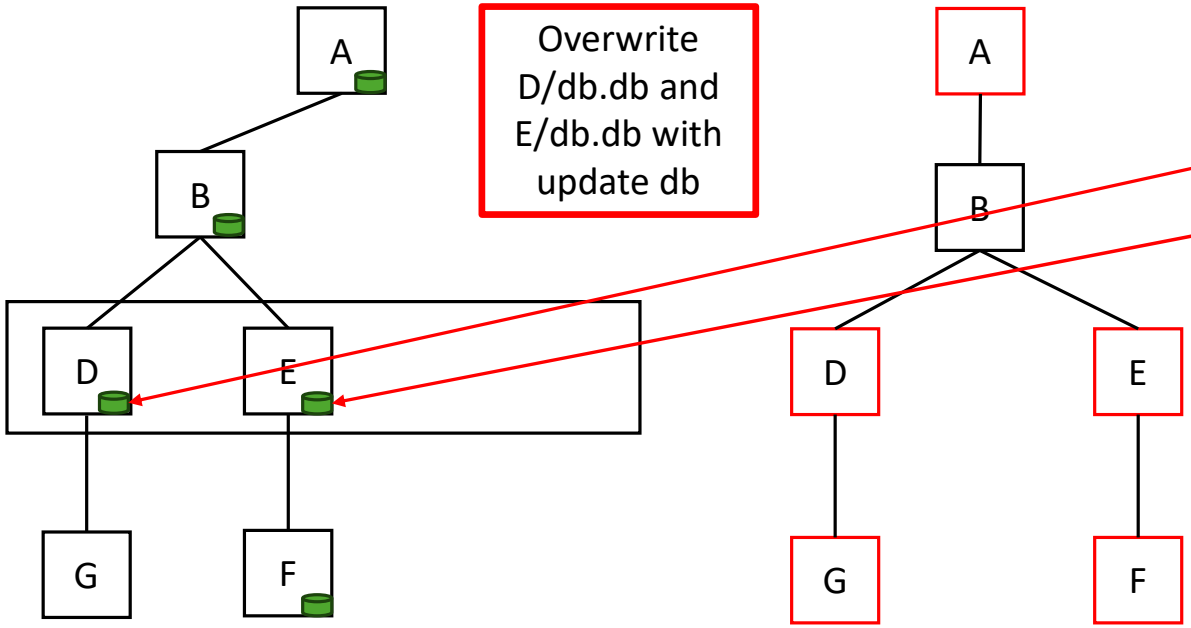
Apply db.db Updates

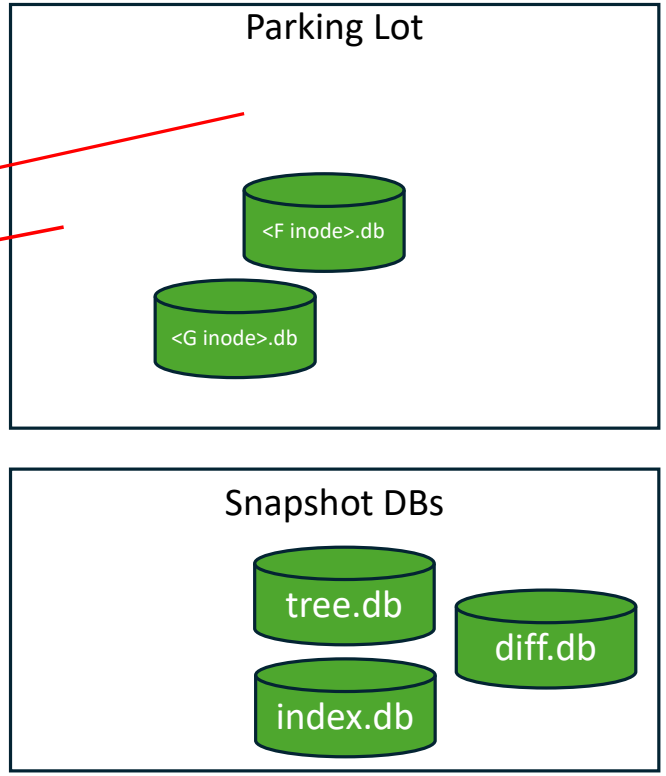
```
SELECT
        indexpath,
        tree path,
        CASE WHEN  indexpath ==  treepath THEN 0 ELSE 1 END,
        tree inode
        tree pinode,
        CASE WHEN  indexpinode ==  treepinode THEN 0 ELSE 1 END
FROM  INCR_DIFF
WHERE  treepath IS NOT NULL
ORDER BY  treedepth ASC;
```

Order doesn't matter

GUFI Tree (Index) (Time 0)

Source Filesystem (Time 1)





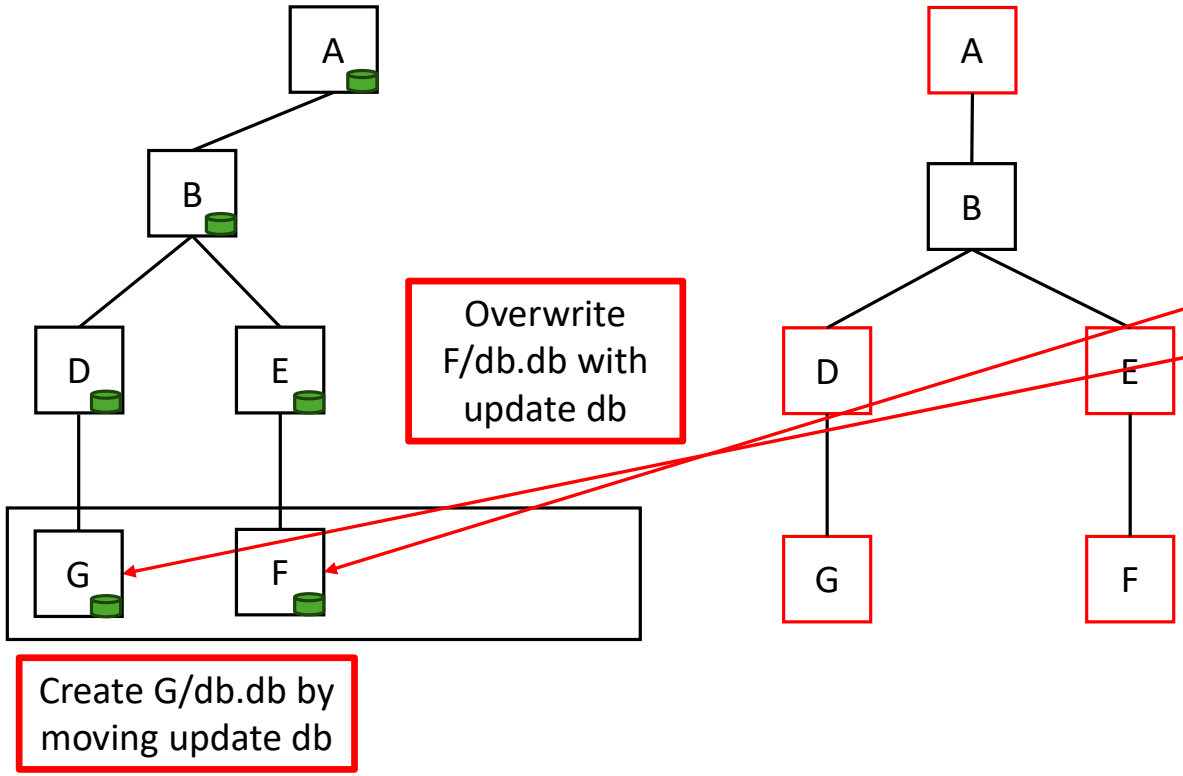
Apply db.db Updates

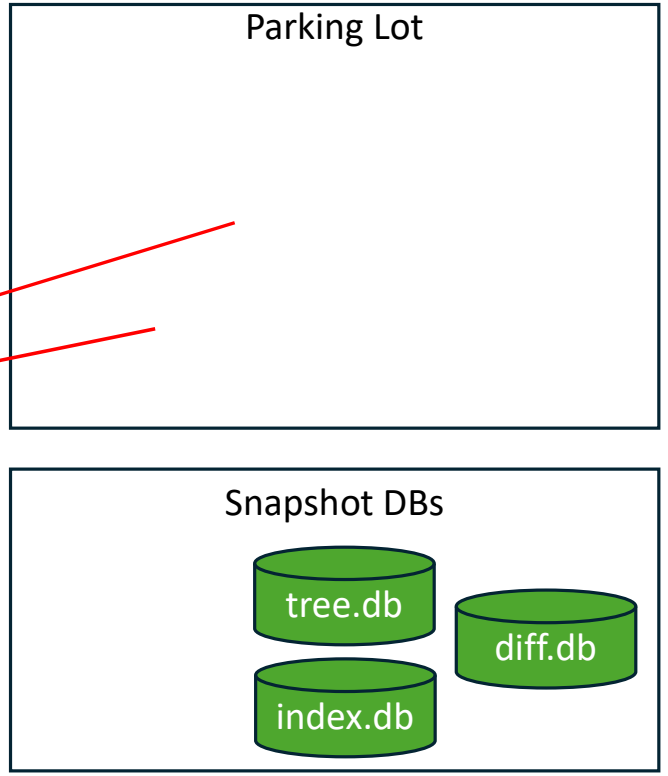
```
SELECT
        indexpath,
        tree path,
        CASE WHEN  indexpath ==  treepath THEN 0 ELSE 1 END,
        tree inode
        tree pinode,
        CASE WHEN  indexpinode ==  treepinode THEN 0 ELSE 1 END
FROM  INCR_DIFF
WHERE  treepath IS NOT NULL
ORDER BY  treedepth ASC;
```

Order doesn't matter

GUFI Tree (Index) (Time 0)

Source Filesystem (Time 1)





The grand vision (Incremental with Replace or Parking Lot